

QUESTION 4(a)

Question:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
GNU nano 8.7.1 skill4a.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50

struct Token
{
    char value[MAX_TOKEN_LENGTH];
};

int main()
{
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("==== Token Parser ==== \n");
    printf("Enter a command: ");

    fgets(input, MAX_INPUT, stdin);

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    int i = 0;

    while (input[i] != '\0')
    {
        /* Skip whitespace */
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')
```

```

/* Create token */
if (token_count >= MAX_TOKENS)
{
    printf("Error: Too many tokens.\n");
    break;
}

int j = 0;

while (input[i] != '\0' &&
        !isspace(input[i]) &&
        input[i] != '|' &&
        input[i] != ';')
{
    if (j < MAX_TOKEN_LENGTH - 1)
    {
        tokens[token_count].value[j++] = input[i];
    }

    i++;
}

tokens[token_count].value[j] = '\0';

if (j > 0)
    token_count++;
}

/* Validate token stream */
printf("\n==== Token Stream =====\n");

if (token_count == 0)
{

```

```
/* Validate token stream */
printf("\n==== Token Stream =====\n");

if (token_count == 0)
{
    printf("Empty input. No tokens found.\n");
    return 0;
}

for (int k = 0; k < token_count; k++)
{
    printf("Token %d: [%s]\n",
           k + 1,
           tokens[k].value);
}

printf("\nTotal Tokens: %d\n", token_count);

/* Debug output */
printf("\n==== Debug Parsing Output =====\n");

for (int k = 0; k < token_count; k++)
{
    if (strcmp(tokens[k].value, "|") == 0)
        printf("Token %d is a PIPE delimiter.\n", k + 1);
    else if (strcmp(tokens[k].value, ";") == 0)
        printf("Token %d is a COMMAND delimiter.\n", k + 1);
    else
        printf("Token %d is a WORD: %s\n",
               k + 1,
               tokens[k].value);
}

return 0;
}
```

Output

```
sri_dhruthi@Dhruthi:~$ nano skill4a.c
sri_dhruthi@Dhruthi:~$ gcc skill4a.c -o skill4a
sri_dhruthi@Dhruthi:~$ ./skill4a
===== Token Parser =====
Enter a command: ls -l | grep txt

===== Token Stream =====
Token 1: [ls]
Token 2: [-l]
Token 3: [|]
Token 4: [grep]
Token 5: [txt]

Total Tokens: 5

===== Debug Parsing Output =====
Token 1 is a WORD: ls
Token 2 is a WORD: -l
Token 3 is a PIPE delimiter.
Token 4 is a WORD: grep
Token 5 is a WORD: txt
sri_dhruthi@Dhruthi:~$ |
```

QUESTION 4(b)

Question:

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_LENGTH 50

struct Node
{
    char value[MAX_LENGTH];
    struct Node *left;
    struct Node *right;
};

struct Node *create_node(char *value)
{
    struct Node *node = malloc(sizeof(struct Node));

    if (node == NULL)
    {
        printf("Memory allocation failed.\n");
        exit(1);
    }

    strcpy(node->value, value);
    node->left = NULL;
    node->right = NULL;

    return node;
}

void print_tree(struct Node *root, int level)
{
    if (root == NULL)
        return;

```

```

void print_tree(struct Node *root, int level)
{
    if (root == NULL)
        return;

    for (int i = 0; i < level; i++)
        printf(" ");

    printf("|-- %s\n", root->value);

    print_tree(root->left, level + 1);
    print_tree(root->right, level + 1);
}

void free_tree(struct Node *root)
{
    if (root == NULL)
        return;

    free_tree(root->left);
    free_tree(root->right);

    free(root);
}

int tokenize(char *input, char tokens[][MAX_LENGTH])
{
    int count = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')

```

```

while (input[i] != '\0')
{
    while (isspace(input[i]))
        i++;

    if (input[i] == '\0')
        break;

    if (count >= MAX_TOKENS)
        return -1;

    int j = 0;

    while (input[i] != '\0' &&
           !isspace(input[i]) &&
           input[i] != '|')
    {
        if (j < MAX_LENGTH - 1)
            tokens[count][j++] = input[i];

        i++;
    }

    tokens[count][j] = '\0';

    if (j > 0)
        count++;

    if (input[i] == '|')
    {
        if (count >= MAX_TOKENS)
            return -1;
    }
}

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skill4b.c
sri_dhruthi@Dhruthi:~$ gcc skill4b.c -o skill4b
sri_dhruthi@Dhruthi:~$ ./skill4b
===== Simple Command Parser =====
Enter command: ls -l

Tokens:
1: ls
2: -l

Syntax is valid.

===== Parse / Execution Tree =====
|-- COMMAND
    |-- ls
    |-- -l

Execution structure generated successfully.
sri_dhruthi@Dhruthi:~$ ./skill4b
===== Simple Command Parser =====
Enter command: ls | grep txt

Tokens:
1: ls
2: |
3: grep
4: txt

Syntax is valid.

===== Parse / Execution Tree =====
|-- PIPE
    |-- COMMAND
        |-- ls
    |-- grep

Execution structure generated successfully.

```

```

sri_dhruthi@Dhruthi:~$ ./skill4b
===== Simple Command Parser =====
Enter command: ls |

Tokens:
1: ls
2: |
Syntax Error: Command cannot end with '|'.
Parsing failed.
sri_dhruthi@Dhruthi:~$

```